Министерство цифрового развития, связи и массовых коммуникаций РФ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

КУРСОВАЯ РАБОТА

Вариант: 18
ТЕМА РАБОТЫ: Дневник питания и тренировок.

Выполнил студент группы ИП-111 Гердележов Даниил Дмитриевич
Проверил доцент кафедры ПМиК к.т.н Мерзлякова Екатерина Юрьевна

г. Новосибирск, 2023

Оглавление

Введение: 3

1. Анализ задач и пользователей 5

2. Выбор репрезентативных задач 7

3. Заимствование 8

4. Прототип интерфейса 10

5. Реализация 13

Функционал: 13

Окна приложения: 13

Структура базы данных: 16

GOMS-анализы: 17

CWT-анализы: 19

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 20

ПРИЛОЖЕНИЕ 21

main.py 21

connection.py 27

Введение:

Область программ и приложений позволяющих следить человеку за своим здоровьем занимают значительную часть среди всех существующих программных продуктов. Современный образ жизни ставит перед людьми задачу поддержания правильного питания и физической активностью, что приводит к росту популярности подобных приложений.

Приложения, представляющие из себя дневник питания, позволяют пользователям учитывать потребляемые продукты, отслеживать калорийность пищи, белковый, жировой и углеводный баланс.

Приложения для тренировок помогают планировать и отслеживать физическую активность, записывать выполненные упражнения, отслеживать время тренировок и прогресс в достижении целей. Самые технологичные из них предоставляют информацию о различных упражнениях, создают персонализированные тренировочные программы и могут даже предложить пользователю мотивационные бонусы или достижения за выполнение определенных заданий.

Таким образом, электронные дневники питания и тренировок помогают людям осознанно вести здоровый образ жизни, предоставляя инструменты для анализа, планирования и отслеживания основных аспектов, влияющих на их физическое и психологическое благополучие.

Выбор данной темы обусловлен растущим интересом общества к здоровому образу жизни и важности систематизации данных о питании и физической активности. Разработка приложения, позволяющего удобно и эффективно вести учет потребляемых продуктов, тренировок, а также фиксировать прогресс в достижении здоровых целей, является хорошим способом закрепить навыки в создании UI/UX.

На сегодняшний день существует ряд приложений, ориентированных на ведение дневника питания и тренировок. Среди наиболее популярных можно выделить приложения для мобильных устройств, такие как MyFitnessPal, Lifesum, Fitbit и другие. Эти приложения предоставляют возможность

пользователям отслеживать свой рацион, фиксировать уровень физической активности и анализировать прогресс в достижении здоровых целей.

1. Анализ задач и пользователей

Для программы "FitApp" представим два типичных пользователя, предполагая разнообразие профессиональных и личных потребностей:

1. **Девушка, 20 лет, заботящаяся о здоровье:**

   Характеристики:

   - Возраст: 20 лет;
   - Интерес к здоровому образу жизни;
   - Возможно, имеет опыт использования подобных приложений или интересуется ими.

   Ожидания от приложения:

   - Простота использования;
   - Визуальная привлекательность;
   - Отслеживание пищевого рациона и физических активностей.

2. **Фитнесс-тренер:**

   Характеристики:

   - Возраст: 30 лет;
   - Профессиональный интерес к фитнесу и тренировкам.

   Образование и навыки:

   - Возможно, образование или сертификация в области физической подготовки;
   - Профессиональный опыт в фитнесе и тренировках;
   - Высокий уровень знаний о питании и тренировках.

   Ожидания от приложения:

   - Функции отслеживания прогресса тренировок и питания;
   - Аналитика и статистика результатов;
   - Возможность планирования индивидуальных программ для клиентов заранее (например, на следующую неделю);

- Удобство использования для самостоятельного контроля и для работы с клиентами.

Основные термины пользователя:

- Энергетическая ценность продуктов питания (калорийность) — расчетное количество тепловой энергии (измеряемое в калориях или джоулях), которое вырабатывается организмом человека или животных при усвоении (катаболизме) съеденных продуктов.
- БЖУ – это белки, жиры, углеводы в рамках суточной калорийности.
- Спортивная тренировка — осмысленная физическая деятельность, направленная на развитие силы, выносливости, ловкости, техничности, скорости и других физических и психологических навыков.

Возможности существующих программ по схожей тематике:

- **GymUp:** Дневник тренировок. Позволяет вести запись результатов тренировки, имеет в себе справочник тренировочных программ и справочник упражнений, поддерживает составление собственных тренировочных программ.
- **iGym Pro:** Дневник тренировок. Позволяет вести запись результатов тренировок, составлять расписания.
- **Nutrilio:** Дневник питания. Позволяет вести журнал питания и употребления воды. Имеет push уведомления – напоминания о воде. Позволяет вести контроль веса пользователя.

2. Выбор репрезентативных задач

1. **Добавление данных о приемах пищи:**

   Запись информации о приемах пищи: вид продуктов, количество потребленных калорий, БЖУ, дополнительную информацию (например размер порции).

2. **Учет физических активностей:**

   Ввод данных о тренировках: тип упражнения, продолжительность, интенсивность.

   Возможность указания использованного оборудования или спортивных инструментов.

3. **Составление плана питания и тренировок:**

   Создание индивидуальных планов по питанию и физическим активностям на определенный период времени (неделя, месяц).

4. **Анализ данных и отслеживание прогресса:**

   Генерация статистики по потребляемым калориям в день.

5. **Использования для хранения данных нескольких пользователей без утраты функциональности.**

   Возможность сохранения данных в сразу нескольких пользователей в уникальных таблицах.

3. Заимствование

iGym Pro:

Дневник тренировок. Позволяет вести запись результатов тренировок, составлять расписания.

Из данного приложения будет позаимствована система планирования и выбора журналов на определённою дату, представленная месяцем календаря.

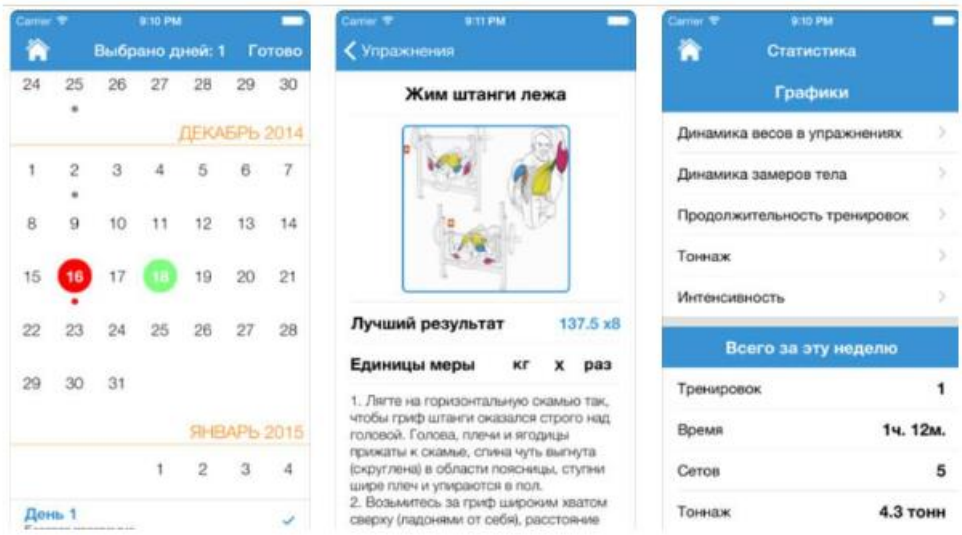


Рисунок 1. – Окна приложения «iGym Pro».

Nutrilio; Diary Nutrition и другие приложения позволяющие вести контроль питания:

Во всех, без исключения, дневниках питания есть одна общая деталь - таблица, которая содержит в себе данные о названии продукта, его пищевой ценности БЖУ на 100 грамм. Аналогично подобным приложениям в моем также будет сводня таблица питания на день.

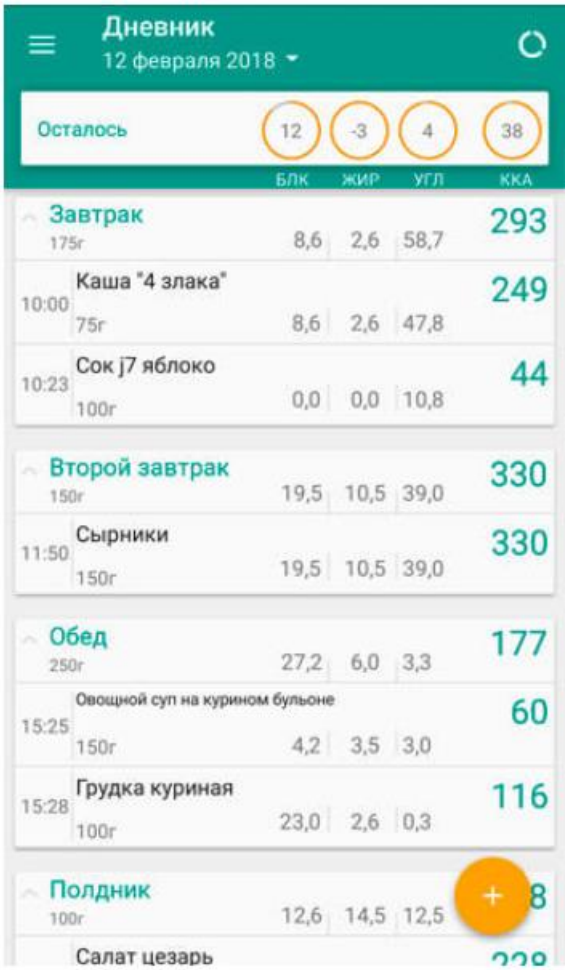


Рисунок 2. – Таблица питания в приложении «Diary Nutrition».

4. Прототип интерфейса

Описание элементов интерфейса:

- Главное окно.

  Главное окно можно разделить на 4 зоны:

  1. MenuBar – включающий управление журналами и справку
  2. Зона календаря – QcalendarWidget предназначенный для выбора даты.
  3. Зона журнала тренировок QTableView предназначенный для отображения данных из базы данных, 2 QPushButtn – добавление и удаление данных.
  4. Зона журнала питания QTableView предназначенный для отображения данных из базы данных, QLable для отображения суммы ккал за день и 2 QPushButtn – добавление и удаление данных.

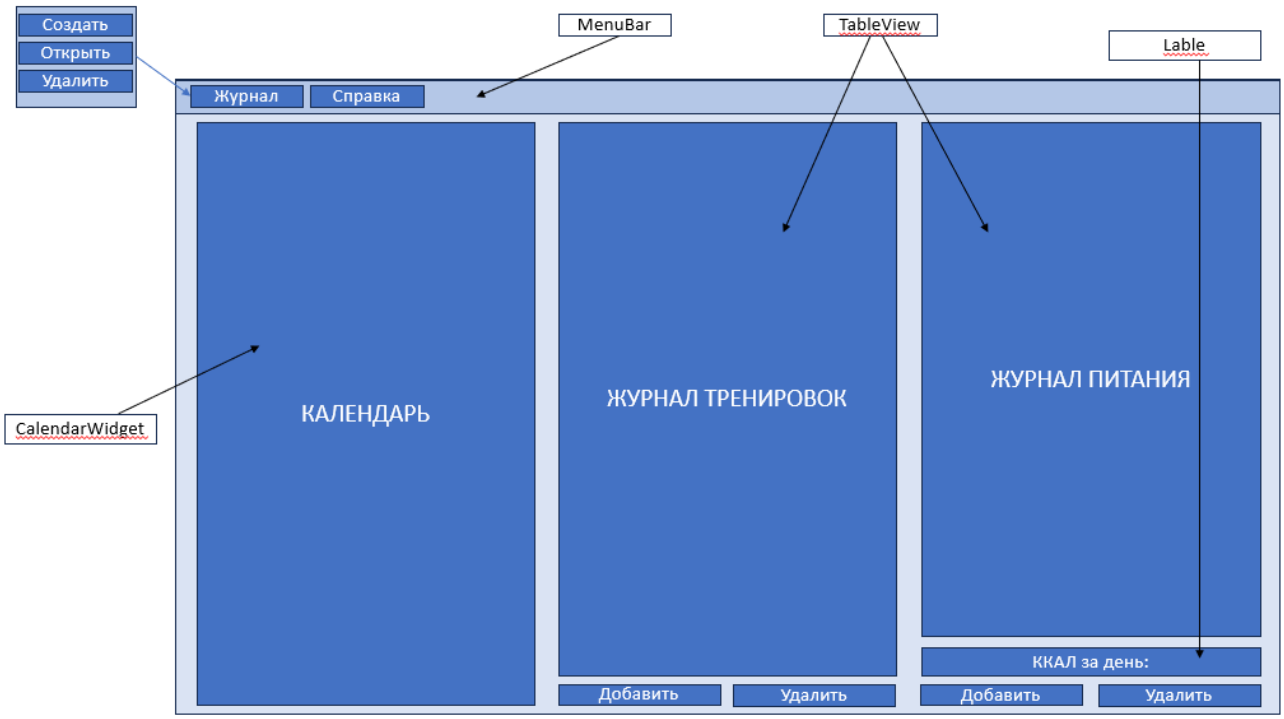


Рисунок 3. Прототип главного окна приложения.

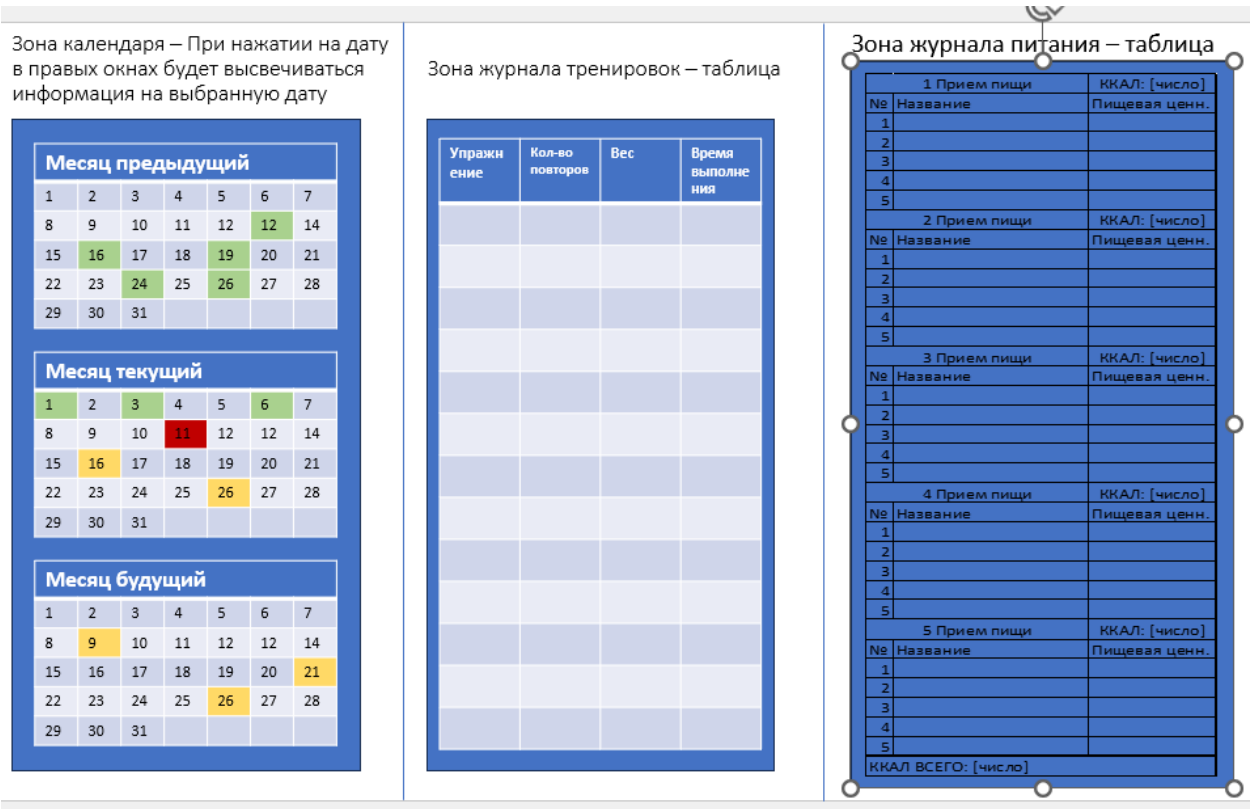


Рисунок 4. Прототип каждой зоны главного окна приложения.

- Окно справки.

  QLable с описанием программы.



Рисунок 5. Прототип окна справки.

- Окна добавления данных.

  Включают в себя несколько QLineEdit для ввода данных и QPushButton для сохранения (подтверждения ввода).

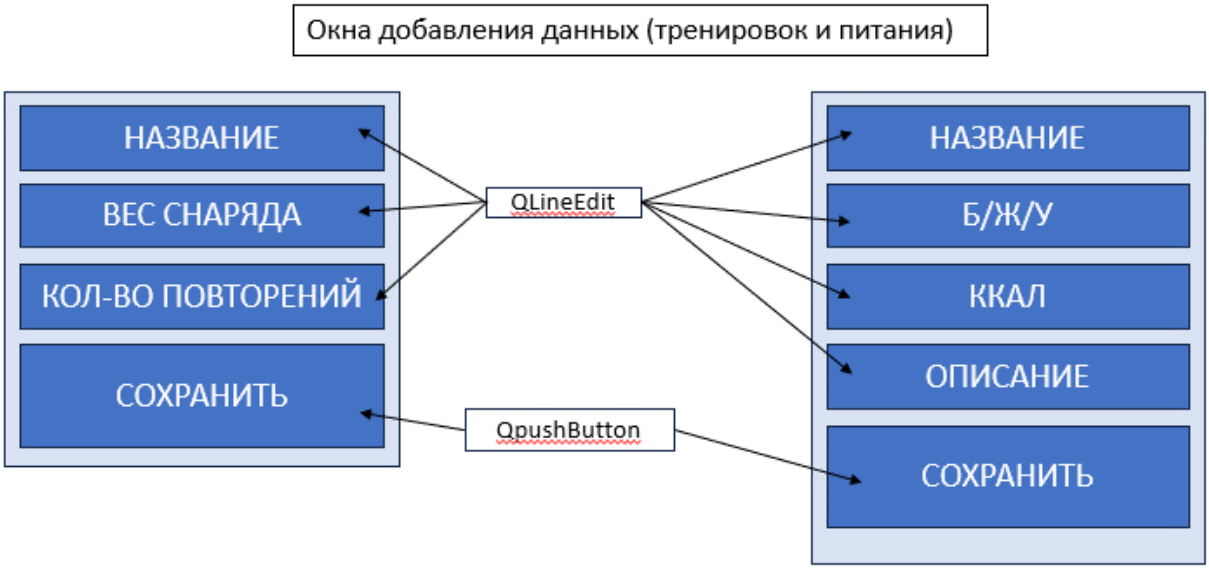


Рисунок 6. Прототипы окон ввода данных.

5. Реализация

Реализация программы "FitApp":

Функционал:

1. **Добавление данных о приемах пищи:**

   Запись информации о приемах пищи: вид продуктов, количество потребленных калорий, БЖУ, дополнительную информацию (например размер порции).
2. **Учет физических активностей:**

   Ввод данных о тренировках: тип упражнения, продолжительность, интенсивность.

   Возможность указания использованного оборудования или спортивных инструментов.
3. **Составление плана питания и тренировок:**

   Создание индивидуальных планов по питанию и физическим активностям на определенный период времени (неделя, месяц).
4. **Анализ данных и отслеживание прогресса:**

   Генерация статистики по потребляемым калориям в день.
5. **Использования для хранения данных нескольких пользователей без утраты функциональности.**

   Возможность сохранения данных в сразу нескольких пользователей в уникальных таблицах.

Окна приложения:

1. **Main (Главное окно):**

   Класс: main

   Главное окно содержит:

   1. Меню – включающее управление журналами и справку
   2. Зона календаря – предназначена для выбора даты.

3. Зона журнала тренировок и две кнопки: добавление и удаление данных.
4. Зона журнала питания текстовое отображение суммы ккал за день и две кнопки добавление и удаление данных.

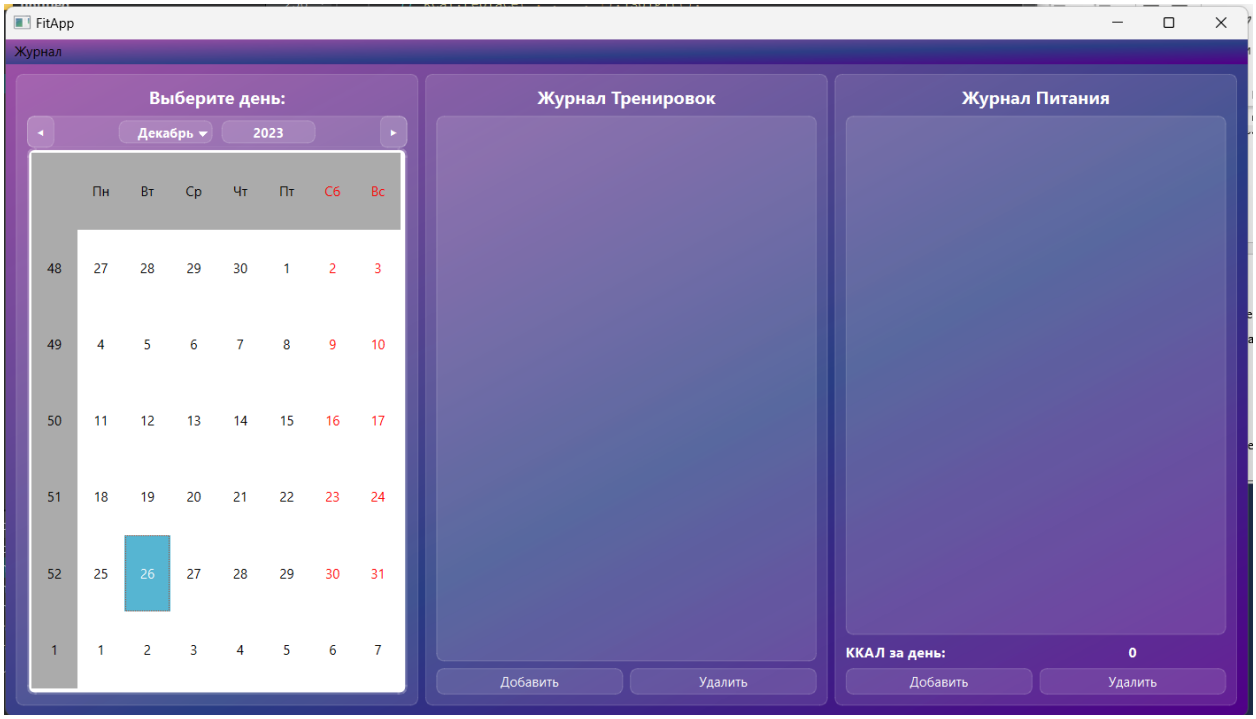


Рисунок 7. Главное окно.

2. **reference (Окно "Справка"):**

- Простое окно с информацией о приложении.

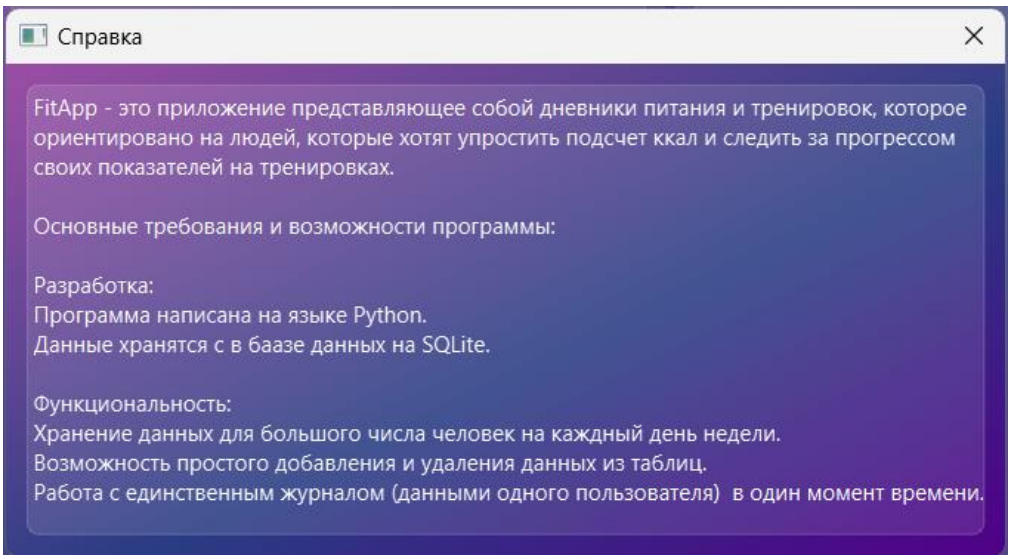


Рисунок 8. Окно справки.

3. **choose_db (Окно "Выбор журнала"):**

- Окно, в котором можно создать новый журнал, открыть или удалить существующий.

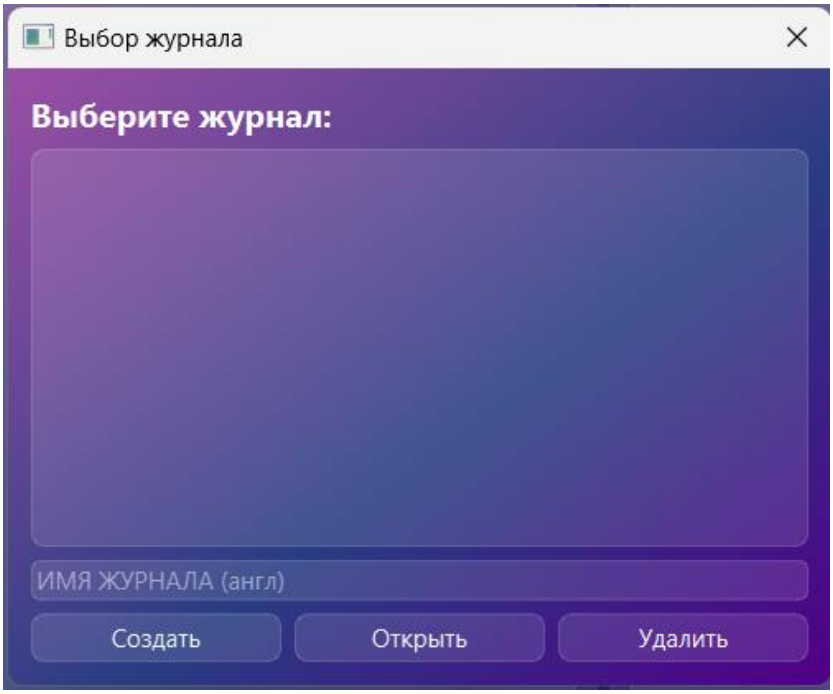


Рисунок 9. Окно выбора журнала.

4. **new_foodStr (Окно "Добавление блюда"):**

- Окно, в котором осуществляется ввод новых данных для добавления блюда в таблицу питания.

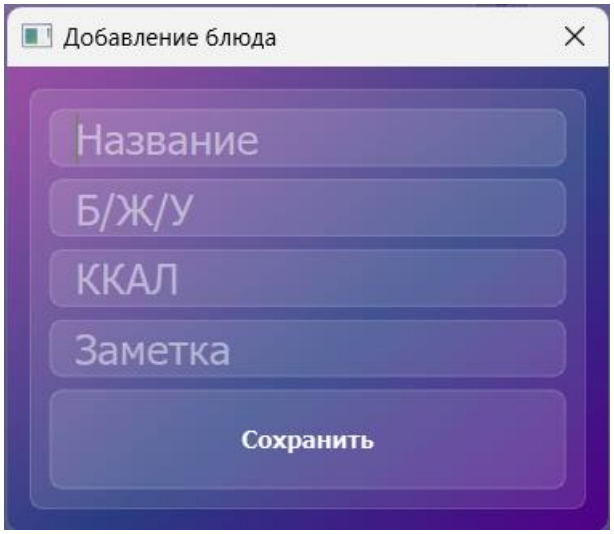


Рисунок 10. Окно добавления данных в таблицу питания.

5. **new_sportStr (Окно "Добавление упражнения"):**

- Окно, в котором осуществляется ввод новых данных для добавления упражнения в таблицу тренировок.

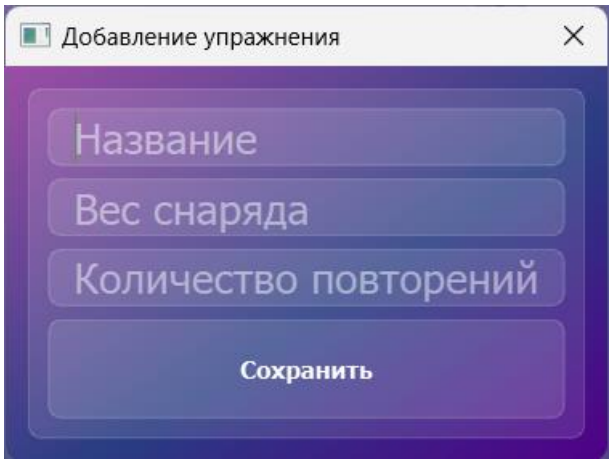
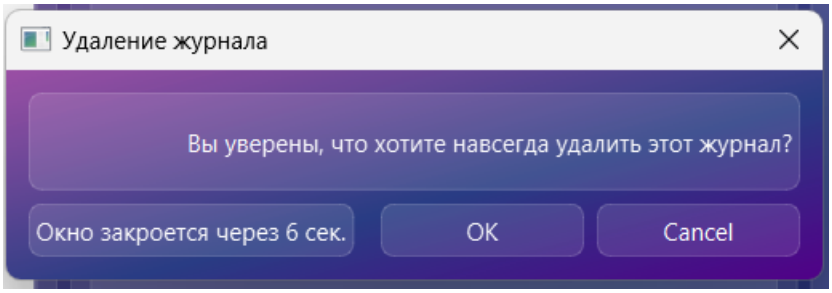


Рисунок 11. Окно Добавления данных в таблицу тренировок.

6. rem_confirm (Окно “Удаление журнала”)



Структура базы данных:

База данных включает в себя три типа таблиц:

1. users:

   Содержит имена журналов.

   Столбцы:

   id INTEGER PRIMARY KEY AUTOINCREMENT

   username VARCHAR(32)

2. user + "_food_" + date:

   Содержит данные таблицы питания для журнала с именем user на дату date.

Столбцы:

id INTEGER PRIMARY KEY AUTOINCREMENT

foodname VARCHAR(16)

bgy VARCHAR(8)

kcal REAL

description VARCHAR(16)

3. user + "_sport_" + date

Содержит данные таблицы тренировок для журнала с именем user на дату date.

Столбцы:

id INTEGER PRIMARY KEY AUTOINCREMENT

taskname VARCHAR(16)

weight REAL

taskcount VARCHAR(10)

Таблица 1 создается при иниализации базы данных, таблицы 2 и 3 после выбора (создания журнала) и при выборе другой даты.

GOMS-анализы:

Задача 1: Создание нового журнала:

Действия необходимые для создания нового журнала:

1) Запустить программу (1.5 с).
    a. Поиск ярлыка (K 0.2 с).
    b. Наведение указателя мыши (P 1.1 с)
    c. Клик по ярлыку (B 0.2 с).

2) Щелчком ЛКМ (левой кнопки мыши) кликнуть по пункту меню “Журнал”, затем кликнуть по подпункту пункту меню “Файл” (3 с).
    a. Поиск кнопки “Журнал” (K 0.2 с).
    b. Наведение указателя мыши (P 1.1 с)
    c. Клик по кнопке (B 0.2 с).
    d. Поиск кнопки “Файл” (K 0.2 с).
    e. Наведение указателя мыши (P 1.1 с)
    f. Клик по кнопке (B 0.2 с).

3) Перед пользователем появилось окно создания журнала (появляется моментально):

   Пользователь должен нажать пункт ввести название в предназначенное для этого поле, ввести имя и нажать на кнопку “Создать” (5.55 с).

   a. Поиск поля ввода (K 0.2 с).
   b. Наведение указателя мыши (P 1.1 с)
   c. Клик по полю ввода (B 0.2 с).
   d. Придумывание названия журнала (M 1.35 с).
   e. Ввод названия журнала (K*6 1.2 с)
   f. Поиск кнопки “Создать” (K 0.2 с).
   g. Наведение указателя мыши (P 1.1 с)
   h. Клик по кнопке (B 0.2 с).

   Затраченное время: 10,05 с.

Задача 2: Добавление записи в таблицу питания:

Действия необходимые для добавления записи в таблицу питания:

1) Щелчком ЛКМ (левой кнопки мыши) кликнуть по кнопке “Добавить” (1,5 с).

   a. Поиск кнопки “Добавить” (K 0.2 с).
   b. Наведение указателя мыши (P 1.1 с)
   c. Клик по кнопке (B 0.2 с).

2) Перед пользователем появилось окно редактирования справочника (появляется моментально):

   Пользователь должен нажать пункт ввести название в предназначенное для этого поле, ввести имя и нажать на кнопку “Создать” (12.1 с).

   a. Поиск поля ввода (K 0.2 с).
   b. Наведение указателя мыши (P 1.1 с)
   c. Клик по полю ввода (B 0.2 с).
   d. Ввод названия блюда (K*8 1.6 с)

   e. Поиск поля ввода (K 0.2 с).
   f. Наведение указателя мыши (P 1.1 с)
   g. Клик по полю ввода (B 0.2 с).
   h. Ввод БЖУ (K*8 1.2 с)

   i. Поиск поля ввода (K 0.2 с).

j. Наведение указателя мыши (P 1.1 с)
k. Клик по полю ввода (B 0.2 с).
l. Ввод ккал (K*3 0.6 с)

m. Поиск поля ввода (K 0.2 с).
n. Наведение указателя мыши (P 1.1 с)
o. Клик по полю ввода (B 0.2 с).
p. Ввод описания (K*6 1.2 с)

q. Поиск кнопки “Сохранить” (K 0.2 с).
r. Наведение указателя мыши (P 1.1 с)
s. Клик по кнопке (B 0.2 с).

Затраченное время: 13,6 с.

CWT-анализы:

Проблема 1:

При запуске программы пользователю никак не показывается необходимость создания (открытия) журнала для работы в приложении.

Решение:

Затемнить и/или деактивировать элементы главного окна до момента открытия журнала.

Проблема 2:

При удалении записи из таблицы пользователю необходимо сначала нажать на клетку id в нужной строке.

Решение:

Добавить диалоговое окно-подсказку, которое высвечивается если пользователь не выбрал ячейку id в нужной строке.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Мерзлякова Е. Ю. Визуальное программирование и человеко-машинное взаимодействие : практикум // Сибирский государственный университет телекоммуникаций и информатики. Новосибирск, 2022. 49 с.
2. Qt, Tools for Each Stage of Software Development Lifecycle [Электронный ресурс]. URL: https://www.qt.io (дата обращения: 02.012.2023).
3. Официальная документация Qt [Электронный ресурс]. URL: https://doc.qt.io (дата обращения: 01.12.2023).
4. Лутц, Марк. Л86 Изучаем Python, том 1, 5-е изд. // Пер. с англ. — СПб.: ООО “Диалектика”, 2019. — 832. ISBN 978-5-907144-51-4 (рус., многотом.).
5. PySide6 6.6.1 [Электронный ресурс]. URL: https://pypi.org/project/PySide6/ (дата обращения: 02.12.2023).

ПРИЛОЖЕНИЕ

main.py

```
import sys

from PySide6 import QtWidgets
from PySide6.QtWidgets import QApplication, QMainWindow, QHeaderView,
QMessageBox, QDialogButtonBox
from PySide6.QtSql import QSqlTableModel
from PySide6.QtCore import Qt, QTimer

from ui_main import Ui_MainWindow
import ui_new_foodStr
import ui_new_sportStr
import ui_choose_db
import ui_rem_confirm
import ui_reference
from connection import Data


class main(QMainWindow):
    def __init__(self):
        super(main, self).__init__()
        self.ui = Ui_MainWindow()
        self.ui.setupUi(self)
        self.user = ""
        self.rm_usrname = ""

        # инициализация стартовых таблиц на сегодняшнюю дату
        self.conn = Data()
        self.choosendate = self.on_date_selected()
        self.conn.create_connection()

        # вывод содержимого таблиц
        self.view_data()
        self.kcalToday()

        # подключение кнопок и действий
        self.ui.db_menu.triggered.connect(self.open_DBwin)
        self.ui.reference_win.triggered.connect(self.open_reference)

        if self.user == "":
            self.ui.btn_new_food.setEnabled(False)
            self.ui.btn_rem_food.setEnabled(False)
            self.ui.btn_new_sport.setEnabled(False)
            self.ui.btn_rem_sport.setEnabled(False)
        self.ui.btn_new_food.clicked.connect(self.open_new_food_window)
        self.ui.btn_rem_food.clicked.connect(lambda: self.delete_current_string(
            self.user + "_food_" + self.on_date_selected()))
```

```
        self.ui.btn_new_sport.clicked.connect(self.open_new_sport_window)
        self.ui.btn_rem_sport.clicked.connect(lambda: self.delete_current_string(
            self.user + "_sport_" + self.on_date_selected()))

        # заполениние журналов данными с выбранной даты
        self.ui.calendarWidget.clicked.connect(self.print_newDay)

    # возвращает дату выбранную на календаре
    def on_date_selected(self):
        selected_date = self.ui.calendarWidget.selectedDate()
        return selected_date.toString("dd_MM_yyyy")

    # Окно выбора журнала (пользователя)
    def open_DBwin(self):
        self.new_window = QtWidgets.QDialog()
        self.ui_window = ui_choose_db.Ui_Dialog()
        self.ui_window.setupUi(self.new_window)
        self.viewUsers()
        self.new_window.show()

        self.ui_window.DBnames_LW.itemClicked.connect(self.on_item_clicked)

        self.ui_window.chooseUser_bttn.clicked.connect(self.choose_usr)
        self.ui_window.addUser_bttn.clicked.connect(self.add_new_usr)
        self.ui_window.remUser_bttn.clicked.connect(self.remove_usr)
        # self.ui_window.saveUser_bttn.clicked.connect(self.saveAs_usr)

    # Окно справки
    def open_reference(self):
        self.new_window = QtWidgets.QDialog()
        self.ui_window = ui_reference.Ui_Dialog()
        self.ui_window.setupUi(self.new_window)
        self.new_window.show()

    # заполнение listWidget именами журналов (пользователей)
    def viewUsers(self):
        self.ui_window.DBnames_LW.clear()
        usr_model = QSqlTableModel(self)
        usr_model.setTable('users')
        usr_model.select()
        for row in range(usr_model.rowCount()):
            username = usr_model.record(row).value('username')
            self.ui_window.DBnames_LW.addItem(username)

    # заполнение LineEdit текстом строки на которую кликнули в listWidget
    def on_item_clicked(self, item):
        self.ui_window.choosen_DB.setText(item.text())

    # выбор существующего журнала (пользователя)
    def choose_usr(self):
```

```
        username_buff = self.ui_window.choosen_DB.text()
        if username_buff != "" and self.conn.check_username_exists(username_buff)
== 1:
            self.user = username_buff
            self.ui.btn_new_food.setEnabled(True)
            self.ui.btn_rem_food.setEnabled(True)
            self.ui.btn_new_sport.setEnabled(True)
            self.ui.btn_rem_sport.setEnabled(True)

            self.view_data()
            self.kcalToday()
            self.new_window.close()

    # создание нового журнала (пользователя)
    def add_new_usr(self):
        username = self.ui_window.choosen_DB.text()

        self.conn.add_new_str_query(
            None, None, None, None, username, "users", self.on_date_selected())
        self.viewUsers()
        self.view_data()
        self.kcalToday()
        self.choose_usr()

    # удаление журнала (пользователя) и всех связанных с ним таблиц
    def remove_usr(self):
        username_buff = self.ui_window.choosen_DB.text()
        if username_buff != "" and self.conn.check_username_exists(username_buff)
== 1:
            self.rm_usrname = username_buff

            self.new_window = QtWidgets.QDialog()
            self.ui_window = ui_rem_confirm.Ui_Dialog()
            self.ui_window.setupUi(self.new_window)
            self.new_window.show()

            self.ui_window.button_box.accepted.connect(self.accept_action)
            self.ui_window.button_box.rejected.connect(self.reject_action)

            # Обратный отсчет - 5 секунд
            self.remaining_time = 9

            # Обновление метки с обратным отсчетом
            self.update_countdown()
            self.timer = QTimer(self)
            self.timer.timeout.connect(self.update_countdown)
            self.timer.start(1000)  # 1000 миллисекунд = 1 секунда

    def update_countdown(self):
        self.ui_window.countdown.setText(
            f"Окно закроется через {self.remaining_time} сек.")
```

```
        self.remaining_time -= 1
        if self.remaining_time < 0:
            self.timer.stop()
            self.reject_action()

    def accept_action(self):
        self.timer.stop()

        self.conn.rem_tables(self.rm_usrname)
        self.user = ""
        self.ui.btn_new_food.setEnabled(False)
        self.ui.btn_rem_food.setEnabled(False)
        self.ui.btn_new_sport.setEnabled(False)
        self.ui.btn_rem_sport.setEnabled(False)

        self.view_data()
        self.kcalToday()
        self.new_window.close()

    def reject_action(self):
        self.timer.stop()
        self.new_window.close()

    """
    def saveAs_usr(self):
        self.add_new_usr()
        username = self.ui_window.choosen_DB.text()
        self.user
        self.conn.saveAS_tables(self.user, username)

        self.viewUsers()
        self.view_data()
        self.kcalToday()
        self.new_window.close()
    """

    # Сумма каллорий за день
    def kcalToday(self):
        self.ui.current_balance.setText(self.conn.kcalSum(
            self.user + "_food_" + self.on_date_selected()))

    # заполнение TableView данными на выбранную дату
    def print_newDay(self):
        # если на выбранной дате нет тоблиц, то зодаем их
        self.conn.add_new_str_query(
            None, None, None, None, self.user, "", self.on_date_selected())
        self.view_data()
        self.kcalToday()

    # вывод содержимого таблиц
    def view_data(self):
```

```
        if (self.user != ""):
            food_model = QSqlTableModel(self)
            food_model.setTable(self.user + "_food_" + self.on_date_selected())
            food_model.select()
            # изменение названий колонок
            food_model.setHeaderData(0, Qt.Horizontal, "№")
            food_model.setHeaderData(1, Qt.Horizontal, "Название")
            food_model.setHeaderData(2, Qt.Horizontal, "Б/Ж/У")
            food_model.setHeaderData(3, Qt.Horizontal, "ККАЛ")
            food_model.setHeaderData(4, Qt.Horizontal, "Заметка")
            self.ui.foodTableView.setModel(food_model)
            header = self.ui.foodTableView.horizontalHeader()  # изменение ширины
колонок
            header.setSectionResizeMode(
                0, QHeaderView.ResizeMode.ResizeToContents)
            header.setSectionResizeMode(1, QHeaderView.ResizeMode.Stretch)
            header.setSectionResizeMode(
                2, QHeaderView.ResizeMode.ResizeToContents)
            header.setSectionResizeMode(
                3, QHeaderView.ResizeMode.ResizeToContents)
            header.setSectionResizeMode(4, QHeaderView.ResizeMode.Stretch)

            sport_model = QSqlTableModel(self)
            sport_model.setTable(self.user + "_sport_" +
                                 self.on_date_selected())
            sport_model.select()
            sport_model.setHeaderData(0, Qt.Horizontal, "№")
            sport_model.setHeaderData(1, Qt.Horizontal, "Название")
            sport_model.setHeaderData(2, Qt.Horizontal, "Вес")
            sport_model.setHeaderData(3, Qt.Horizontal, "Кол-во повторов")
            self.ui.sportTableView.setModel(sport_model)
            header = self.ui.sportTableView.horizontalHeader()
            header.setSectionResizeMode(
                0, QHeaderView.ResizeMode.ResizeToContents)
            header.setSectionResizeMode(1, QHeaderView.ResizeMode.Stretch)
            header.setSectionResizeMode(
                2, QHeaderView.ResizeMode.ResizeToContents)
            header.setSectionResizeMode(
                3, QHeaderView.ResizeMode.ResizeToContents)

    # открытие окон для добавления записей в таблицы
    def open_new_food_window(self):
        self.new_window = QtWidgets.QDialog()
        self.ui_window = ui_new_foodStr.Ui_Dialog()
        self.ui_window.setupUi(self.new_window)
        self.new_window.show()
        self.ui_window.btn_new_food.clicked.connect(self.add_new_foodStr)

    def open_new_sport_window(self):
        self.new_window = QtWidgets.QDialog()
        self.ui_window = ui_new_sportStr.Ui_Dialog()
```

```
        self.ui_window.setupUi(self.new_window)
        self.new_window.show()
        self.ui_window.btn_new_sport.clicked.connect(self.add_new_sportStr)

    # добавления записей в таблицы
    def add_new_foodStr(self):
        foodname = self.ui_window.le_foodname.text()
        bgy = self.ui_window.le_bgy.text()
        kcal = self.ui_window.le_kcal.text()
        description = self.ui_window.le_description.text()

        if kcal.replace('.', '', 1).isdigit():
            self.conn.add_new_str_query(
                foodname, bgy, kcal, description, self.user, "_food_",
self.on_date_selected())
            self.view_data()
            self.kcalToday()
            self.new_window.close()

    def add_new_sportStr(self):
        taskname = self.ui_window.le_taskname.text()
        weight = self.ui_window.le_weight.text()
        count = self.ui_window.le_count.text()

        self.conn.add_new_str_query(
            taskname, weight, count, None, self.user, "_sport_",
self.on_date_selected())
        self.view_data()
        self.kcalToday()
        self.new_window.close()

    # удаление строки (записи) из журнала
    def delete_current_string(self, tablename):
        if "food" in tablename:
            index = self.ui.foodTableView.selectedIndexes()[0]
            id = str(self.ui.foodTableView.model().data(index))
        elif "sport" in tablename:
            index = self.ui.sportTableView.selectedIndexes()[0]
            id = str(self.ui.sportTableView.model().data(index))

        self.conn.delete_jurnalStr(id, tablename)
        self.view_data()
        self.kcalToday()


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = main()
    window.show()

    sys.exit(app.exec())
```

connection.py

```
from PySide6 import QtWidgets, QtSql


class Data:
    def __init__(self):
        super(Data, self).__init__()

    # первоначальная инициалзация БД
    def create_connection(self):
        db = QtSql.QSqlDatabase.addDatabase('QSQLITE')
        db.setDatabaseName('usr_db.db')

        if not db.open():
            QtWidgets.QMessageBox.critical(None, "Cannot open database",
                                           "Click Cancel to exit.",
QtWidgets.QMessageBox.Cancel)
            return False

        query = QtSql.QSqlQuery()

        query.exec("CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY
AUTOINCREMENT, "
                   "username VARCHAR(32))")

        return True

    # выполнение SQL запросов
    def execute_query_with_params(self, sql_query, query_values=None):
        query = QtSql.QSqlQuery()
        query.prepare(sql_query)

        if query_values is not None:
            for query_value in query_values:
                query.addBindValue(query_value)

        query.exec()

        return query

    # формирование запросов на создание таблиц и их заполнение
    def add_new_str_query(self, var1, var2, var3, var4, user, table, date):
        if user != "":
            query = QtSql.QSqlQuery()

            tablename = user + "_food_" + date
            sql_query = ("CREATE TABLE IF NOT EXISTS {} (id INTEGER PRIMARY KEY
AUTOINCREMENT, "
                         "foodname VARCHAR(16), "
                         "bgy VARCHAR(8), "
```

```
                        "kcal REAL, "
                        "description VARCHAR(16))").format(tablename)
            query.exec(sql_query)

            tablename = user + "_sport_" + date
            sql_query = ("CREATE TABLE IF NOT EXISTS {} (id INTEGER PRIMARY KEY
AUTOINCREMENT, "
                        "taskname VARCHAR(16), "
                        "weight REAL,  "
                        "taskcount VARCHAR(10))").format(tablename)
            query.exec(sql_query)

            tablename = user + table + date
            if "food" in tablename:
                sql_query = "INSERT INTO {} (foodname, bgy, kcal, description)
VALUES (?, ?, ?, ?)".format(
                    tablename)
                self.execute_query_with_params(
                    sql_query, [var1, var2, var3, var4])
            elif "sport" in tablename:
                sql_query = "INSERT INTO {} (taskname, weight, taskcount) VALUES
(?, ?, ?)".format(
                    tablename)
                self.execute_query_with_params(sql_query, [var1, var2, var3])
            elif "users" in tablename:
                sql_query = "INSERT INTO users (username) VALUES (?)"
                self.execute_query_with_params(sql_query, [user])

    # удаление записи (строки) из таблицы по её номеру (id)
    def delete_jurnalStr(self, id, tablename):
        sql_query = "DELETE FROM {} WHERE id=?".format(tablename)
        self.execute_query_with_params(sql_query, [id])

    # сумма ккал за день
    def kcalSum(self, tablename):
        sql_query = "SELECT SUM(kcal) FROM {}".format(tablename)

        query_values = []

        query = self.execute_query_with_params(sql_query, query_values)

        if query.next():
            return str(query.value(0))

        return '0'

    # удаление пользователя из таблицы users и всех таблиц связанных с ним
    def rem_tables(self, username):
        sql_query = f"SELECT name FROM sqlite_master WHERE type='table' AND name
LIKE '{username}%'"
```

```
        # Получаем список таблиц для удаления
        query = self.execute_query_with_params(sql_query)
        tables = []
        while query.next():
            tables.append(query.value(0))

        sql_query = "DELETE FROM users WHERE username = ?"
        self.execute_query_with_params(sql_query, [username])
        # Удаляем найденные таблицы
        for table_name in tables:
            drop_query = f"DROP TABLE IF EXISTS {table_name}"
            self.execute_query_with_params(drop_query)

    # Есть ли пользователь с заданным именем
    def check_username_exists(self, username):
        sql_query = "SELECT COUNT(*) FROM users WHERE username = ?"
        query = self.execute_query_with_params(sql_query, [username])
        if query.next():
            count = query.value(0)
            if count > 0:
                return 1  # Имя пользователя уже существует
        return 0
    """
    def saveAS_tables(self, old_username, new_username):
        # Получаем список таблиц для копирования
        food_tables = []
        sport_tables = []

        sql_query_food = f"SELECT name FROM sqlite_master WHERE type='table' AND
name LIKE '{old_username}_food_%'"
        query_food = self.execute_query_with_params(sql_query_food)
        while query_food.next():
            food_tables.append(query_food.value(0))

        sql_query_sport = f"SELECT name FROM sqlite_master WHERE type='table' AND
name LIKE '{old_username}_sport_%'"
        query_sport = self.execute_query_with_params(sql_query_sport)
        while query_sport.next():
            sport_tables.append(query_sport.value(0))

        # Копируем таблицы с новым именем пользователя
        for table_name in food_tables:
            new_table_name = table_name.replace(old_username, new_username)
            copy_query = f"CREATE TABLE {new_table_name} AS SELECT * FROM
{table_name}"
            self.execute_query_with_params(copy_query)

        for table_name in sport_tables:
            new_table_name = table_name.replace(old_username, new_username)
            copy_query = f"CREATE TABLE {new_table_name} AS SELECT * FROM
{table_name}"
```

```
        self.execute_query_with_params(copy_query)
"""
```